

课程项目设想：基于 SysY 的分层 IR、Value/Memory SSA 与可视化编译器

0. 项目概述

本项目以 SysY 风格的类 C 语言为前端输入，目标是实现一条完整、可验证、可扩展的编译器主线，并把其中的中间表示（IR）设计做成项目的核心特色：

AST -> semantic -> canonical IR -> lower IR -> value SSA -> memory SSA -> SSA passes -> backend/codegen

本项目的重点放在：

1. 设计多层、职责清晰的 IR；
2. 在 lower IR 上显式化 memory/value 边界；
3. 在 value SSA 上实现严格的 dominator-tree 驱动 SSA construction；
4. 继续规划 memory SSA，将 load/store 与内存依赖分析系统化；
5. 在 SSA 之上实现共享 analysis 与一组小而可验证的优化 pass；
6. 在同一语言主线下引入受控的 functional mode，先通过语义约束禁用一部分命令式特性，再逐步扩展少量函数式能力；
7. 增加面向人类理解的可视化输出与解耦分析，包括静态结构图、基于这些图的耦合热点提示，以及类似 gdb 的单步执行/状态观察界面，用来辅助程序解耦、展示优化与重构依据；
8. 如果后续引入指针或更复杂的内存对象，再补一层内存检查/安全分析，用于检测非法访问、未初始化读、悬空引用等问题；
9. 用 verifier、dump、fixed-point regression 和代表性程序族测试来保证中间层的正确性与工程可维护性。

目前项目已经完成前端、canonical IR、lower IR，以及一条严格得多的 lower_ir -> value_ssa conversion 主线；后续重点将从“能否构造 value SSA”转向“如何继续推进 memory SSA、共享分析与更系统的 SSA 优化”。

1. 项目背景

1.1 课程背景

SysY 是编译原理课程中常用的小型教学语言，优点是：

- 语法规模适中；
- 便于实现词法、语法、语义分析；

- 适合逐步引入 IR、优化和后端。

1.2 本项目的切入点

本项目在完整编译器主线之上，以 SysY 为基础，重点探索：

1. 前端到中端的清晰分层

- 从 AST 到 canonical IR；
- 再从 canonical IR 到 lower IR；
- 再从 lower IR 到 value SSA；
- 再从 value SSA 继续走向 memory SSA 与后续后端。

2. 显式 memory/value 边界

- 将局部变量、全局变量视为 slot；
- 将计算值统一落到 temp / SSA value 上；
- 用 `load_*` / `store_*` 显式表达读写。

3. 严格、可验证的 SSA construction

- 基于 dominator tree、phi placement、rename scope 的严格转换；
- 尽量避免“过渡态兜底逻辑”长期滞留在主线。

4. 分层 SSA 与中端优化实验

- 在 canonical IR 做较轻的 source-near 优化；
- 在 value SSA 层做更结构化、更可组合的 SSA-native pass；
- 在后续 memory SSA 层探索更系统的 load/store 与内存依赖优化。

5. 语言 profile 与函数式扩展实验

- 不把项目拆成两套完全不同的语言；
- 而是在同一套前端/IR 主线上增加 functional mode；
- 第一阶段先通过 semantic restriction 禁用一部分命令式能力，再视进度逐步加入少量函数式特性。

6. 编译过程可视化与面向人的解耦分析

- 把已有 analysis 结果转成更适合课程展示的图形输出；
- 并补一条偏动态的执行可视化路线，用于单步观察程序状态变化；
- 包括函数调用关系、全局依赖关系、CFG、SSA 和 dominator/phi 结构图；
- 进一步面向人类读者总结哪些函数/全局关系过于耦合，帮助理解程序结构。

7. 安全性补强（规划中）

- 如果后续语言侧引入指针、引用或更复杂的内存对象；
- 可以在现有 verifier / IR analysis 之外，再增加一层 memory checking；
- 这层检查可以与解释器/执行器共用执行语义，在单步执行时同步做动态检查；
- 用于支持更强的安全性检查与错误诊断展示。

1.3 当前已实现的扩展/特点

相对最小 SysY 教学实现，本项目当前已经具备一些更强的能力：

- 更完整的表达式与控制流 lowering；

- 顶层 global / declare / runtime-init helper 机制；
- canonical IR verifier 与 pass pipeline；
- 独立 lower IR 层；
- 独立 value SSA 层；
- 已经为后续 memory SSA 留出清晰的 lower/value 分层基础；
- strict dominator-tree-driven SSA construction；
- 多种 SSA-side cleanup / fold / normalization / forwarding pass。

接下来计划继续补强的两条课程展示线是：

- functional mode
- 可视化输出

因此这份项目的特色可以概括为：

以 SysY 为输入载体，构建完整编译器，并把“分层 IR + SSA + 静态/动态可视化”做成主线亮点

2. 项目设计

2.1 输入与输出

输入

项目输入是 SysY 风格的源程序。规划上将支持两个 language profile：

1. 默认模式 (imperative / SysY-like)

- 保持现在这条命令式主线；
- 允许赋值、循环、更新表达式、全局写入等。

2. 函数式模式 (functional mode)

- 第一阶段不引入完整函数式语言，而是先做“受限函数式 profile”；
- 通过文件头声明或编译参数启用；
- 先在语义层禁止一部分命令式能力，例如：
 - 变量重赋值
 - ++/--
 - compound assignment
 - 某些可变全局写入
 - 视项目进度限制部分循环结构
- 第二阶段再考虑加入少量函数式特有功能（如 let、更表达式化绑定、或受限的一等函数能力）。

当前前端已经能够完成：

- 词法分析；
- 语法分析；
- 语义检查；
- 生成 canonical IR。

中间输出

项目当前最重要的输出其实是分层 IR：

1. canonical IR

- 非 SSA；
- 更接近源码语义；
- 支持 canonical pass pipeline。

2. lower IR

- 将 local/global 从 value 中拆出；
- 显式引入 load_local/store_local/load_global/store_global ；
- 保持 CFG 基本形状稳定。

3. value SSA

- 在 lower IR 之上只对 value flow 做 SSA；
- 引入 phi ；
- 保留 load/store ，不做 memory SSA。

4. memory SSA（规划中）

- 在 value SSA 之后，进一步把 memory state 的流动也规范化；
- 为后续 dead store、load/store forwarding、内存依赖优化提供更系统的基础；
- 这一层目前还是主线规划，不属于已完成部分。

最终输出

当前项目已经具备较强的“IR 输出与验证”能力，但还没有进入完整 backend/codegen 阶段。

因此本课程项目的阶段性输出目标是：

- 正确、稳定的 IR dump；
- verifier-legal 的 canonical / lower / value SSA 程序；
- 多阶段优化后的规范化 IR 结果。

除此之外，还计划提供一类“给人看的输出”：

- 函数调用图；
- 函数与全局变量之间的读写依赖图；
- 代表性 CFG / dominator tree / phi 分布图；
- pass 前后结构变化图或统计摘要；
- 面向人理解的耦合摘要，例如“哪些函数依赖过多全局”“哪些模块边界可能需要重构”；
- 基于某层 IR 解释器的单步执行视图，例如逐步展示当前语句、调用栈、局部变量和关键 IR 状态。

- 如果后续引入更复杂内存对象，还可以基于同一解释器输出空间/内存检查结果。

后续项目将继续往后端推进，目标包括：

- 中间表示到后端表示的衔接；
- 目标代码或目标汇编输出；
- 如有需要再补解释执行器或测试型执行框架。

只是从当前完成度看，backend/codegen 还处在后续阶段，而不是已经完成的部分。

2.2 中间表示分层

(1) canonical IR

作用：

- 作为语义分析后的稳定前端输出面；
- 保留 source-level 结构；
- 支持轻量中端优化。

特点：

- 非 SSA；
- block-based；
- 有严格 verifier；
- 更适合做常量折叠、copy propagation、CFG 清理等轻量 pass。

(2) lower IR

作用：

- 显式化 memory/value 边界；
- 给 SSA construction 提供更干净的输入层；
- 保持 CFG 结构稳定。

特点：

- value refs 只允许 temp/immediate；
- locals/globals 只通过 load/store 出现；
- 已有独立 CFG/dominator/frontier/phi-placement analysis。

(3) value SSA

作用：

- 将 lower IR 中的 value flow 规范化为 SSA；

- 作为后续共享 def/use 分析和 SSA-native pass 的基础层。

特点：

- 只对 value 做 SSA，不做 memory SSA；
- phi 为 block-entry 一等公民；
- strict conversion 主线已完成；
- 后续优化 pass 已独立拆分到 `value_ssa_pass`。

(4) memory SSA（规划中）

作用：

- 在 value SSA 之后继续把 memory state 结构化；
- 更系统地表示 load/store 之间的依赖关系；
- 为更强的内存相关优化和后端衔接打基础。

特点：

- 不是简单重复 value SSA，而是面向 memory effect；
- 预期会消费 lower IR 与 value SSA 之间已经分清的 memory/value 边界；
- 目前还处在规划阶段，不计入已完成功能。

2.3 pass 设计

A. canonical IR pass

当前 canonical IR 已有一条默认 pass pipeline，典型包含：

- immediate fold；
- temp constant propagation；
- temp copy propagation；
- simplify-cfg；
- dead-temp elimination。

这些 pass 的目标是：

- 保持 IR 语义正确；
- 尽量简化 source-near 结构；
- 为 lower IR 降低一部分噪声。

B. lower IR analysis

这层不是优化 pass，而是 strict SSA construction 的输入分析层，当前已包括：

- CFG analysis;
- predecessor / successor;
- reachability;
- dominator / immediate dominator;
- dominance frontier;
- temp definition blocks;
- temp phi candidate lists;
- live-in 分析;
- pruned phi candidate lists;
- successor-phi-use lists;
- dominator walk。

其作用是：

- 为 lower_ir -> value_ssa conversion 提供“放 phi、按什么顺序 rename、哪些 predecessor 边需要填值”的严格输入事实。

C. value SSA conversion

当前 conversion 已经不再是早期那种“边猜边补”的过渡路径，而是严格得多的主线：

1. 基于 lower-IR-side analysis 预创建 pruned candidate phi;
2. 按 dominator tree DFS;
3. block entry 绑定 phi result;
4. block 内 use/def 只通过 rename scope 解析;
5. predecessor leave 直接填 successor phi incoming;
6. finalize 只做 completeness check;
7. 最后做 alpha-renaming 以获得稳定、稠密的 SSA id。

D. value SSA shared analysis

当前已经有共享 def/use 分析：

- def-site metadata;
- use counts;
- explicit use-site lists。

这是后续 SSA-side pass 的公共基础设施。

E. value SSA pass

目前已经独立拆到：

- include/value_ssa_pass.h

- `src/value_ssa_pass/`

当前已落地的 pass 包括：

1. `value_ssa_simplify_trivial_values`
2. `value_ssa_forward_local_loads`
3. `value_ssa_forward_global_loads`
4. `value_ssa_normalize_binary_operands`
5. `value_ssa_simplify_algebraic_identities`
6. `value_ssa_fold_constants`
7. `value_ssa_eliminate_redundant_binaries`
8. `value_ssa_simplify_cfg`
9. `value_ssa_eliminate_dead_value_defs`
10. `value_ssa_eliminate_dead_stores`
11. `value_ssa_canonicalize_program`

这些 pass 的定位是：

- 先做窄而可解释的 SSA-native cleanup / optimization；
- 以共享 analysis 为基础，而不是每个 pass 自己重扫；
- 暂不进入 memory SSA、寄存器分配、复杂 alias analysis。

F. memory SSA / memory-aware optimization（规划中）

这一步预期承接当前 lower IR 显式 `load/store` 与 value SSA 的成果，重点考虑：

- 哪些 memory state 需要被显式建模；
- 哪些局部 slot 可以进一步脱离 memory 语义；
- 如何让后续 dead store、memory forwarding、内存依赖分析更系统。

这一层和 value SSA 不完全重复：

- value SSA 处理 value definition/use；
- memory SSA 处理 memory state 与 memory effect。

G. functional mode（规划中）

functional mode 的目标不是立刻把语言改造成完整函数式语言，而是：

1. 在同一门语言下引入一个受限的 profile；
2. 尽量复用现有 parser / semantic / IR 主线；
3. 先通过语义约束禁用部分命令式特性；
4. 再视时间逐步扩展少量函数式特有功能。

建议的阶段划分是：

阶段 1：约束型函数式

- 增加 mode 开关（文件头声明或编译参数）；
- 在 semantic 层禁止：
 - 变量重赋值；
 - ++/-- ；
 - compound assignment；
 - 某些可变全局写入；
 - 视进度限制部分循环结构；
- IR 主线尽量保持不分叉。

阶段 2：少量函数式特性

- 引入更表达式化的绑定形式；
- 视工作量讨论是否加入：
 - 不可变 let
 - 受限的一等函数引用
 - 有限 lambda / 闭包支持

当前更稳妥的课程路线是：

先做函数式约束，再考虑少量函数式特性

H. 可视化（规划中）

可视化部分的目标不是替代编译主线，而是同时覆盖两类展示能力：

- 静态结构可视化：把已有 analysis / IR 结构转成更适合课程展示和程序理解的图形输出；
- 动态执行可视化：提供类似 gdb 的单步执行与状态观察界面，帮助展示程序在运行过程中的控制流和数据变化。

在此基础上，再进一步产出面向人的解耦分析结论。

优先考虑的可视化包括：

1. 函数调用图

- 节点表示函数
- 边表示调用关系
- 用于展示模块耦合与解耦空间

2. 函数-全局依赖图

- 展示哪个函数读/写哪个 global
- 用于辅助理解状态传播与潜在副作用

3. CFG / dominator / SSA 结构图

- 展示 lower IR / value SSA 的结构

- 用于解释 strict SSA construction、phi 分布和优化前后结构变化

4. pass 前后对照图

- 展示 canonicalization 或特定优化 pass 前后的结构差异
- 更偏“给人看的优化”

5. 解耦分析摘要

- 基于调用图和全局依赖图给出耦合热点提示
- 例如哪些函数依赖过多、哪些 global 被过多位置读写、哪些模块边界可以进一步拆分

6. 单步执行可视化

- 提供类似 gdb 的执行界面
- 可以按语句、按 basic block，或按某层 IR 进行 step
- 底层依赖一层解释器或执行器来维护程序状态，而不是直接从编译过程“推断”运行结果
- 第一版更适合建立在 lower IR 之上，因为控制流、load/store 和 slot 状态都更明确
- 展示当前位置、调用栈、局部变量、关键 SSA/value/memory 状态

输出形式可以分阶段：

- 第一阶段：dot/graphviz + 基于解释器的 execution trace 展示
- 第二阶段：更交互式的网页可视化与单步执行界面

I. 内存检查 / 安全分析（规划中）

这一部分不是当前 SysY 主线的必需项，但如果后续扩展到指针、引用或更复杂的内存对象，它将成为很自然的一条补强线。

可以考虑的能力包括：

- 非法内存访问检查；
- 未初始化内存读取检查；
- 悬空引用或越界访问的保守检测；
- 将检查结果和 IR / CFG / 调用关系可视化结合起来做课程展示。

这一层和优化 pass 不完全相同，它更偏向：

- 运行前的静态安全分析；
- 或者基于解释器/执行器的动态空间检查与执行时诊断；
- 或者带插桩的检查型 lowering / backend 辅助能力。

在实现路径上，这一层和单步执行可视化可以共用同一套执行语义：

- 解释器负责维护当前控制流位置、调用栈、局部变量和内存对象状态；
- 检查器在 load/store 或后续更复杂的内存访问点上附加合法性检查；
- 可视化界面再把错误、警告和对象状态变化展示出来。

2.4 canonicalization pipeline

当前已经有一条稳定的 SSA canonicalization pipeline，用于：

- 稳定 IR dump；
- 做 fixed-point regression；
- 给后续测试与比较提供规范形。

当前 pipeline 已经包含：

- trivial simplification；
- forwarding；
- normalization；
- algebraic identities；
- constant folding；
- redundant-binary cleanup；
- DCE；
- CFG cleanup；
- alpha-renaming。

其设计原则是：

- 小步；
- 保守；
- 可验证；
- 便于 fixed-point 检查。

3. 实现方案

3.1 工作量估计

从当前实际完成情况看，本项目已经完成了一个较大规模的基础设施部分：

- 前端 (lexer / parser / semantic)；
- canonical IR；
- lower IR；
- value SSA skeleton；
- strict conversion；
- 一部分 SSA-side pass。

因此后续工作量主要集中在中后段：

1. 持续扩充 shared SSA analysis;
2. 增加更系统的 SSA pass;
3. 推进 memory SSA 与 memory-aware optimization;
4. 增加 functional mode;
5. 增加可视化输出;
6. 为动态可视化补一层面向某层 IR 的解释器/执行器;
7. 如果语言后续扩到指针等特性, 再补内存检查/安全分析;
8. 继续完成 backend/codegen, 并整理 benchmark、实验报告和结果分析。

如果按课程项目的粒度估计:

- 当前已完成部分足以构成一个较完整的“实现主体”;
- 后续更像是中后期优化、实验和收尾。

3.2 代码结构

当前项目代码结构已经比较清晰:

前端

- src/lexer/
- src/parser/
- src/semantic/

canonical IR

- include/ir.h
- src/ir/
- src/ir_pass/

lower IR

- include/lower_ir.h
- src/lower_ir/
- tests/lower_ir/

value SSA core

- include/value_ssa.h
- src/value_ssa/
- tests/value_ssa/

value SSA pass

- include/value_ssa_pass.h

- src/value_ssa_pass/

memory SSA / backend（后续规划）

- 预期在现有 lower_ir / value_ssa 分层之上继续扩展；
- 具体目录结构将在 memory SSA 设计与 backend 落地时最终拍板。

这种结构的优点是：

- 表示层和 pass 层分开；
- lower IR 和 value SSA 分开；
- verifier / dump / conversion / pass 都有独立模块；
- 便于之后做维护、实验和扩展。

3.3 工程策略

本项目的实现策略不是“一次性做完全部编译器”，而是：

1. 先做 verifier-backed 的表示层；
2. 再做严格、可测试的转换；
3. 再在结果层之上做共享分析；
4. 最后再叠加 SSA-native pass。

这套策略的核心优点是：

- 每一层都有稳定 contract；
- 每一层都能通过 regression 锁行为；
- 出问题时容易定位是在：
 - lowering
 - verifier
 - conversion
 - pass哪一层。

4. 预期效果

4.1 功能性目标

本项目预期至少达到以下效果：

1. 能将 SysY 风格输入程序稳定 lowering 到 canonical IR；

2. 能再 lowering 到 lower IR，并显式化 load/store ；
3. 能再 strict conversion 到 value SSA；
4. 能进一步向 memory SSA 主线推进，系统表示 memory-side 优化所需事实；
5. 能在 SSA 上执行一组基础优化 pass；
6. 所有阶段输出都能通过各自 verifier；
7. 代表性输入在 canonicalization 后能达到 fixed point；
8. 在 functional mode 下，语义层能稳定拒绝被禁用的命令式特性；
9. 能生成调用关系、依赖关系、CFG/SSA 的可视化结果；
10. 能基于调用图和依赖图输出面向人的解耦分析摘要；
11. 能基于某层 IR 解释器提供类似 gdb 的单步执行可视化，用于观察调用栈、变量和值状态变化；
12. 如果后续引入指针等能力，能补上基础的内存检查/安全分析；
13. 后续能继续衔接 backend/codegen。

4.2 代表性程序族

当前已经通过大量 regression 锁住的代表性例子包括：

- straight-line 程序；
- diamond CFG；
- same-return diamond；
- simple slot-carried loop；
- loop-carried temp；
- multi-backedge loop；
- multi-carried-temp loop；
- nested-loop carried-temp family；
- runtime-global startup/helper family；
- call-heavy CFG family；
- local/global load-store forwarding family；
- dead-result call / dangerous binary 保守保留 family。

所以本项目的一个重要预期效果，不一定是“跑出最快的汇编”，而是：

能够系统地展示不同 IR 层怎样逐步把 source-near 程序变成显式 memory、再变成 SSA、再在 SSA 上做结构化优化。

对于 functional mode，更适合通过 profile 对照样例展示效果，例如：

- 默认模式允许、函数式模式拒绝的程序；
- 在函数式模式下仍能通过的表达式化程序；
- 如果后续加入 let / 受限函数式特性，再展示对应 lowering 结果。

对于可视化，更适合展示：

- 一个中小型程序的调用图；

- 一个带 global 的程序的读/写依赖图；
- 同一函数在 lower IR / value SSA / canonicalize 之后的结构图对比；
- 一组面向人的解耦提示，例如“某函数依赖过多 global”“某 global 写入点过多”；
- 一个简单程序在 lower IR 解释器上的单步执行界面，展示当前语句、调用栈、局部变量和值变化。

4.3 benchmark 与实验方式

这里有两种可选实验方式：

方案 A：以代表性样例为主

优点：

- 更适合当前项目现状；
- 容易展示 specific feature；
- 更适合当前已经完成的前中端与 SSA 主线。

可以比较：

- canonical IR dump
- lower IR dump
- value SSA dump
- canonicalize 前后对比
- pass 前后对比

方案 B：在后端主线接上之后，引入 benchmark

可以考虑：

- SysY/课程常见 benchmark；
- 自己整理的小型控制流与表达式压力样例；
- 统计：
 - block 数
 - instruction 数
 - phi 数
 - 优化前后 IR 长度
 - canonicalization fixed-point 情况

4.4 当前阶段的预期目标

结合当前项目完成度，现阶段预期效果可以概括为：

1. 在代表性程序族上展示：
 - lower IR 的显式 memory/value 分离；

- value SSA 的严格构造；
 - memory SSA 的规划方向与必要性；
 - SSA-side cleanup / optimization 的效果。
2. 用 regression 和 verifier 证明：
 - 输出合法；
 - fixed-point 稳定；
 - effectful 指令不会被错误删除；
 - 危险 binary 不会被错误折叠。
 3. 用 profile 对照样例证明：
 - functional mode 确实比默认模式更严格；
 - 这些限制不会破坏既有 IR 主线。
 4. 用图形化结果展示：
 - 调用关系
 - 全局依赖
 - CFG / SSA 结构
 5. 如果后续扩展到指针等能力，再展示一组 memory checking / 安全分析样例。
 6. 如果时间允许，再补一层更接近 benchmark 的量化比较。

5. 参考文献

课程报告中实际会用到的参考资料可以收成下面这些：

1. **SysY 语言规范 / 课程教学资料**
 - 用于说明输入语言、语义约束和课程背景。
2. Cytron, R., Ferrante, J., Rosen, B. K., Wegman, M. N., & Zadeck, F. K. (1991).
Efficiently Computing Static Single Assignment Form and the Control Dependence Graph.
ACM TOPLAS.
 - 用于 SSA、支配树、dominance frontier、phi placement 的理论背景。
3. Cooper, K. D., & Torczon, L.
Engineering a Compiler.
 - 用于编译器整体结构、IR 设计、数据流分析与优化框架的参考。
4. LLVM Project Documentation.
 - 用于参考现代 IR、verifier、analysis 与 pass pipeline 的组织方式。
5. LLVM MemorySSA Documentation.
 - 用于参考 memory SSA 的建模方法，以及 load/store 相关优化的组织方式。
6. Peyton Jones, S.
The Implementation of Functional Programming Languages.
 - 用于函数式模式和后续函数式特性扩展的背景参考。
7. Graphviz Documentation.

- 用于调用图、依赖图、CFG 和 SSA 可视化输出的实现参考。

6. 当前已完成内容

1. 已完成词法、语法、语义分析主线；
2. 已完成 canonical IR 与 verifier；
3. 已完成 canonical IR pass pipeline；
4. 已完成 lower IR 设计、lowering、verifier 与代表性 regression；
5. 已完成 value SSA 数据模型、verifier、dump；
6. 已完成严格的 `lower_ir -> value_ssa` dominator-tree construction；
7. 已完成共享 SSA def/use analysis；
8. 已完成一组窄而可解释的 SSA-native pass；
9. 已完成 canonicalization pipeline 与 fixed-point regression。